

ACMP BE DEVELOPER

TECHNICAL HOMEWORK



BRIEF | STORY



The Grand Unified Holding Corp. (GUHC) is an old-school conglomerate from the Kingdom of Rohan. For decades they have managed their customer and partner “accounts” in Excel sheets and email threads.

Each account actually belongs to a hierarchy:

- ▶ *A global account* (top parent)
- ▶ Regional branches
- ▶ Country offices
- ▶ Local resellers

Every decision (discounts, contracts, support) depends on **who** is parent of **whom**, but nobody can see the full structure in one place. Moving a reseller under a different regional office takes days of manual edits and mistakes are common (even cycles in the hierarchy — a child becoming its own grandparent!).

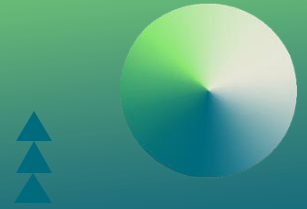
GUHC’s board finally decided to build a small but solid **Hierarchy Accounts System** as a first step toward modernisation. They hired *you* to build the MVP backend service in .NET, plus a tiny console client for the sales team.

REQUIREMENTS



- ▶ GUHC wants a **backend service** that manages account hierarchies in a strict **tree structure**.
- ▶ The system will be later integrated into a larger CRM, so:
 - ▶ It must expose **HTTPS REST APIs**.
 - ▶ It must be easy to discover: **Swagger/OpenAPI** is required.
- ▶ Technical constraints:
 - ▶ **.NET 8** (ASP.NET Core Web API).
 - ▶ **MS SQL Server** as the database.
 - ▶ Use **Git** for file version control.
- ▶ Non-functional expectations:
 - ▶ Tree depth is limited to **5 levels**.
 - ▶ The hierarchy must never contain **cycles**.
 - ▶ Moving accounts around must preserve a valid tree (no “moving the root under its child”).
- ▶ The sales team will use a **lightweight console application** that calls the REST API and prints the hierarchy (or a sub-tree) in the terminal.

ASSIGNMENT – GENERAL



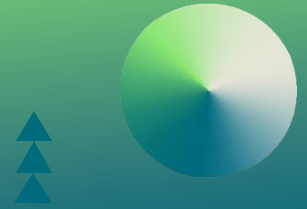
- ▶ Build the Hierarchy Accounts System
- ▶ This is an **individual homework** for a backend developer. Focus on clean design, readability, and testability rather than fancy UI.

Data model & database

- ▶ Design a SQL Server schema to store accounts in a parent-child tree.
- ▶ Requirements:
 - ▶ Support up to 5 levels from root to deepest child.
 - ▶ Prevent cycles in the hierarchy.
- ▶ You may use EF Core or plain SQL – but structure must be clear and documented.
- ▶ **Unit tests**
 - ▶ Cover core business logic (e.g., validation of depth, cycle prevention, move rules, delete behaviour).

- ▶ Source code in a Git repository with:
 - ▶ Small, meaningful commits (per task/feature).
 - ▶ Clear project structure (Api, Domain, Tests, Console, etc.).
- ▶ A short README explaining:
 - ▶ How to run migrations / create DB.
 - ▶ How to start the API and console app.
 - ▶ Example API calls or sample data flow.

ASSIGNMENT – REST API



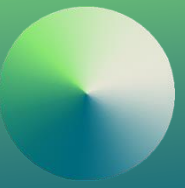
- ▶ Create hierarchy account
 - ▶ Creates a new account under an optional parent.
 - ▶ Validate max depth (no deeper than 5 levels).
 - ▶ Validate that adding this node does not create a cycle.
- ▶ Move hierarchy account to a different parent
 - ▶ Change the parent of an existing account.
 - ▶ Cannot move the top/root account under another node.
 - ▶ Ensure the move does not introduce cycles and keeps depth ≤ 5 .
- ▶ Delete hierarchy account
 - ▶ When deleting an account that has children,
 - its children must be reassigned to the deleted account's parent.
 - ▶ Deleting the top/root account should be forbidden or clearly handled (you decide and document).
- ▶ Get single account
 - ▶ Returns account details (id, name, parent, depth, etc.).
- ▶ Get account tree / subtree
 - ▶ Given an account ID, return the whole subtree rooted at that account (account + all descendants).
 - ▶ Response structure should be hierarchical (e.g., nested children).
- ▶ Swagger / OpenAPI
 - ▶ The APIs must be documented and visible via Swagger UI.

ASSIGNMENT – CONSOLE APPLICATION

⚠ Optional: If you have time / wishes to extend on the assignment 🖱

- ▶ Create a small .NET console app that consumes your REST API.
- ▶ On start, it should:
 - ▶ Accept an optional accountId parameter (e.g., CLI argument).
 - ▶ If no accountId is supplied, print the full tree.
 - ▶ If accountId is supplied, print only the subtree of that account.
- ▶ Output in a readable hierarchical format (indentation or tree-like ASCII).

HEADS UP



- ▶ Through homework, try to explain a bit your experience of SOLID, Clean Code practices, RESTful principles.
Nevertheless, the homework may take enormous amount, we recommend finding fair amount of time to not go too deep in details, but to be able to show good practices for developing simple MVP.

Goal is to see your thinking around:

- ▶ Model hierarchical data,
- ▶ Design and validate business rules,
- ▶ Implement clean .NET APIs,
- ▶ Write tests,
- ▶ And structure a small but complete solution end-to-end.

THANK YOU

